



Chapitre 7

Les Chaînes de caractères



Chaînes de caractères

- Il n'existe pas de type spécial chaîne ou string en C. Une chaîne de caractères est traitée comme un tableau de caractères
- Une chaîne de caractères en C est caractérisée par le fait que le dernier élément vaut le caractère '\0', ceci permet de détecter la fin de la chaîne
- Il existe plusieurs fonctions prédéfinies pour le traitement des chaînes de caractères (ou tableaux de caractères)



Déclaration

Syntaxe : **char** <NomVariable> [<Longueur>]; //tableau de caractères

Exemple : **char** NOM [15];

- Pour une chaîne de N caractères, on a besoin de N+1 octets en mémoire (le dernier octet est réservé pour le caractère '\0')
- Le nom d'une chaîne de caractères est le représentant de l'adresse du 1^{er} caractère de la chaîne
- On peut aussi manipuler les chaînes de caractères en utilisant des pointeurs



Initialisation

- On peut initialiser une chaîne de caractères à la définition :
 - comme un tableau, par exemple : `char ch[ ] = {'e','c','o','l','e','\0'}`
 - par une chaîne constante, par exemple : `char ch[ ] = "école"`
 - en attribuant *l'adresse d'une chaîne de caractères constante* à un pointeur sur char, par exemple : `char *ch = "école"`
- On peut préciser le nombre d'octets à réserver à condition que celui-ci soit supérieur ou égal à la longueur de la chaîne d'initialisation
 - `char ch[ 6] = "école"` est valide
 - `char ch[4] = "école"` ou `char ch[ 5] = "école"` provoque une erreur



Traitement des chaînes de caractères

- Le langage C dispose d'un ensemble de bibliothèques qui contiennent des fonctions spéciales pour le traitement de chaînes de caractères
- Les principales bibliothèques sont :
 - La bibliothèque `<stdio.h>`
 - La bibliothèque `<string.h>`
 - La bibliothèque `<stdlib.h>`
- Nous verrons les fonctions les plus utilisées de ces bibliothèques



Fonctions de la bibliothèque <stdio.h>

- **printf()** : permet d'afficher une chaîne de caractères en utilisant le spécificateur de format %s.

Exemple : **char ch[]= " Bonsoir " ;**
printf(" %s ", ch);

- **puts(<chaine>)** : affiche la chaîne de caractères désignée par <Chaîne> et provoque un retour à la ligne.

Exemple : **char *ch= " Bonsoir " ;**
puts(ch); /*équivalente à printf("%s\n ", ch);*/



Fonctions de la bibliothèque <stdio.h>

- **scanf()** : permet de saisir une chaîne de caractères en utilisant le spécificateur de format %s.

Exemple : **char Nom[15];**
printf("entrez votre nom");
scanf(" %s ", Nom);

Remarque : le nom d'une chaîne de caractères est le représentant de l'adresse du premier caractère de la chaîne, il ne doit pas être précédé de &

- **gets(<chaîne>)** : lit la chaîne de caractères désignée par <Chaîne>

Exemple : **char phrase[100];**
printf("entrez une phrase");
gets(phrase);



Fonctions de la bibliothèque <string.h>

- **strlen(ch)**: fournit la longueur de la chaîne sans compter le '\0' final Exemple : **char s[]="Test";**
printf("%d",strlen(s)); //affiche 4
- **strcat(ch1, ch2)** : ajoute ch2 à la fin de ch1. Le caractère '\0' de ch1 est écrasé par le 1^{er} caractère de ch2

Exemple : **char ch1[20]=" Bonne ", *ch2=" chance ";**
strcat(ch1, ch2) ;
printf(" %s", ch1); // affiche Bonne chance



Fonctions de la bibliothèque <string.h>

- **strcmp(ch1, ch2):** compare ch1 et ch2 lexicographiquement et retourne une valeur :
 - ❑ nul si ch1 et ch2 sont identiques
 - ❑ négative si ch1 précède ch2
 - ❑ Positive si ch1 suit ch2
- **strcpy(ch1, ch2) :** copie ch2 dans ch1 y compris le caractère '\0'
- Exemple : **char ch[10];**
 - **strcpy(ch, " Bonjour ");**
 - **puts(ch);** // affiche Bonjour



Chapitre 8 introduction aux structures de données



Structures



- **Une structure** est un nouveau type de données constitué par un ensemble de variables (champs) qui peuvent être hétérogènes et de types différents
- La différence avec le type tableau est que les champs d'un tableau sont tous homogènes et du même type
- Les structures permettent de représenter des objets réels caractérisées par plusieurs informations, par exemple :
 - **Une personne** caractérisée par son nom (chaîne), son âge (entier), sa taille (réel), ...
 - **Une voiture** caractérisée par sa marque (chaîne), sa couleur (chaîne), son année modèle(entier), ...



Déclaration d'une structure

déclaration d'une structure s'effectue en précisant le nom de la structure, ainsi que le nom et le type de ses champs :

- **Syntaxe en C :**

```
struct nom_structure  
{ type 1 nom_champ1; type 2 nom_champ2;  
...  
type N nom_champN;  
};
```

- **Exemple :**

```
struct Personne  
{ char Nom[20];  
int Age;  
float taille;  
};
```



Déclaration d'une variable structure

- La réservation se fait quand on définit des variables correspondant à ce modèle de structure. Ceci peut se faire soit :

```
struct Personne  
{ char Nom[20];  
  int Age;  
  float taille;  
}
```

- après la déclaration de la structure, par exemple :

```
struct Personne p1, tab[10];
```

//p1 est une variable de type Personne

// tab est une variable de type tableau de 10 éléments (de type Personne)





Initialisation d'une structure

- Lors de la déclaration d'une variable structure, on peut initialiser ses champs avec une notation semblable à celle utilisée pour les tableaux en indiquant la liste des valeurs respectives entre accolades.

- Exemple :

```
struct date
{
    int jour;
    char mois[10];
    int annee;
};
```

```
struct date d1= {15,"Novembre", 2013};
```



Accès aux champs d'une structure

- L'accès à un champ d'une variable structure se fait par le nom de la variable suivi d'un point et du nom du champ

`nom_variable.nom_champ`

Exemple: `p1.age` représente le champ age de la variable structure p1

```
struct Personne
{ char Nom[20];
  int Age;
  float taille;
}
```

- On peut déclarer la personne p1:

```
struct Personne p1 = { "Mohamed", 20, 1.7};
```



Accès aux champs d'une structure : Exemple

Saisie et affichage d'une structure

- Soit la structure article définie par :

struct article

```
{    int numero; //un numéro qui identifie l'article  
    char nom[20];  
    int qte_stock; // quantité disponible en stock  
    float prix;  
};
```

- Soit la déclaration de variables :

struct article art;

Saisissez les champs de la structure art, puis affichez son contenu



Exemple de Saisie et d'affichage d'une structure

```
/* Saisie des champs de la structure art */
```

```
printf ("Entrez respectivement les champs de l'article \n ");
```

```
scanf(" %d %s %d %f" , &art.numero, art.nom, &art.qte_stock, &art.prix);
```

```
/* Affichage des champs de la structure art */
```

```
printf (" Cet article a pour : \n ");
```

```
printf (" \t numéro : %d \n ", art.numero);
```

```
printf (" \t nom : %s \n ", art.nom);
```

```
printf (" \t quantité en stock : %d \n ", art.qte_stock);
```

```
printf (" \t prix : %f \n ", art.prix);
```



Manipulation des structures

- Les structures sont manipulées en général champ par champ. Toutefois, la norme ANSI permet d'affecter une structure à une autre structure de même type, par exemple : `struct Etudiant E1,E2;`

`E2=E1;` est une instruction valide qui recopie tous les champs de E1 dans les champs de E2

- Rq: Il n'est pas possible de comparer deux structures. Les instructions `if(E1==E2)` ou `if(E1!=E2)` ne sont pas permises



Définition de types synonymes: typedef

- En C, on peut définir des types nouveaux synonymes de types existants (simples, pointeur, tableau, structure,...) en utilisant le mot clé **typedef**.
- Ces nouveaux types peuvent être utilisées ensuite comme les types prédéfinis
- **Exemple d'un type synonyme d'un type simple :**
- **typedef int entier;** définit un nouveau type appelé **entier** synonyme du type prédéfini **int**
- **entier i=4,T[10] ;** le type entier est utilisé ici pour déclarer des variables



Exemples de typedef

- Type synonyme d'un type tableau :

typedef float tableau[10];

tableau T; T est un tableau de 10 réels

typedef int matrice[4][5];

matrice A; A est une matrice d'entiers de 4 lignes et 5 colonnes